National University of Computer and Emerging Sciences

DL-2001: Introduction to Data Science
Lab Manual 12

Department of Data Science
FAST-NU, Lahore, Pakistan

Table of Contents

Objectives

After performing this lab, students shall be able to:

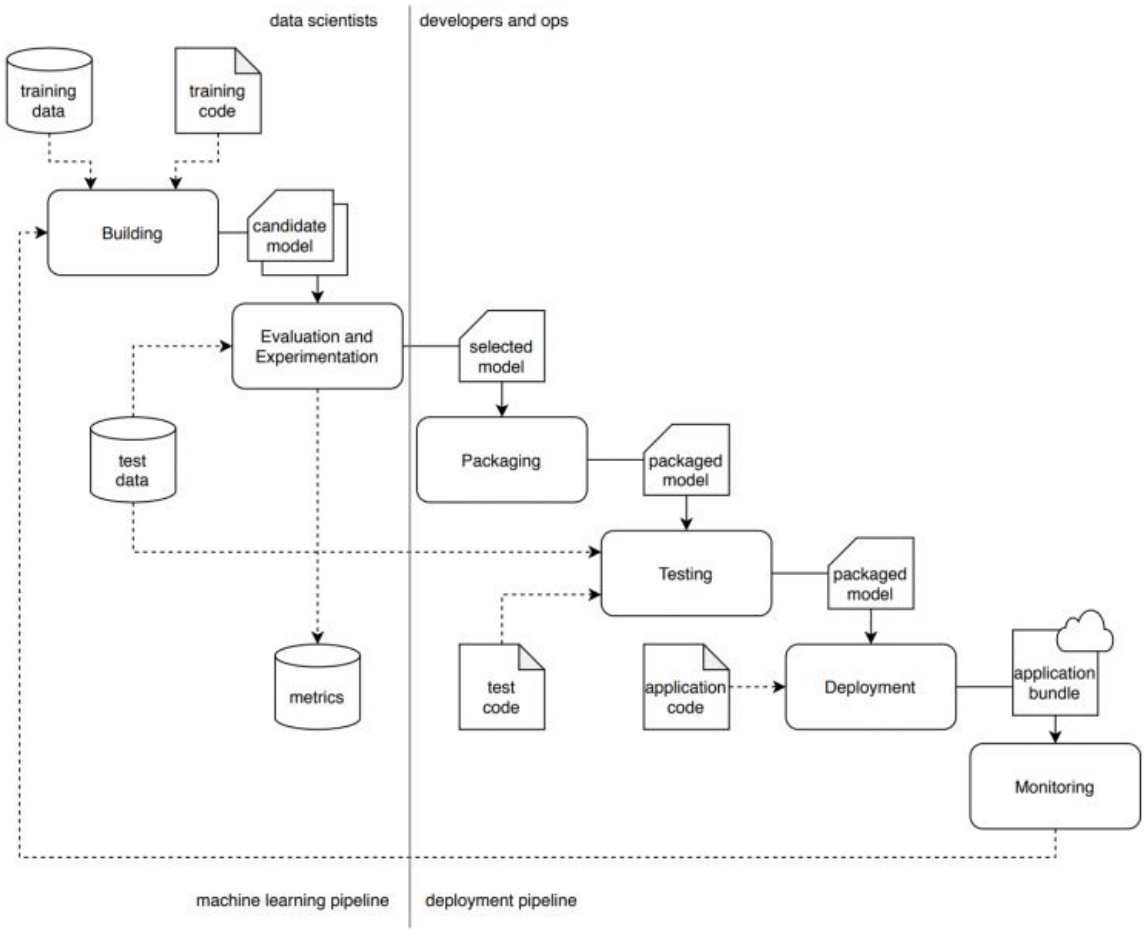
- ✓ Understand different model deployment techniques
- ✓ Make a streamlit application

1 What is Model Deployment?

Deployment means taking that model out of your notebook and making it usable by others, such as:

- a web app
- a mobile app
- an API that another system can call
- or even an IoT device

In short: Model Deployment = Making your trained model available for real-world use.



2 Common Deployment Techniques

Technique	Description	When to Use
Streamlit	Easiest way to turn a model into an interactive web app	Teaching, demos, prototypes
Flask / FastAPI	Expose the model as a REST API	Production systems, integration with other apps
Docker + Cloud (AWS/GCP)	Containerize and deploy model on scalable infrastructure	Production, scaling
Hugging Face Spaces	Host model + Streamlit/Gradio UI for free	Public demos
MLflow / BentoML	Frameworks for versioned model deployment	MLOps, enterprise projects

3 Streamlit

Streamlit is an open-source Python library for building interactive web apps using only Python. It's ideal for creating dashboards, data-driven web apps, reporting tools and interactive user interfaces without needing HTML, CSS or JavaScript.

3.1 Streamlit Installation

Make sure Python and pip are already installed on your system. To install Streamlit, run the following command in the command prompt or terminal:

```
pip install streamlit
```

3.2 How to run a Streamlit file?

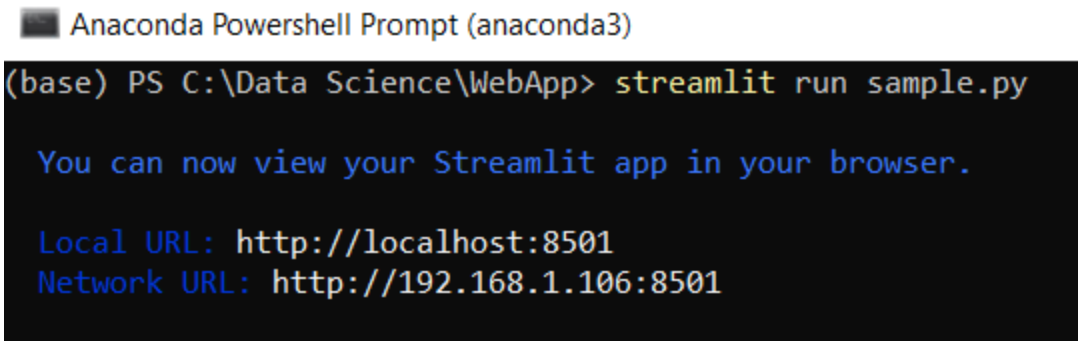
To run a Streamlit app, open the command prompt or Anaconda prompt and type:

```
streamlit run filename.py
```

For example, if file name is sample.py, run:

```
streamlit run sample.py
```

After running the command, Streamlit starts a local development server and displays a URL (http://localhost:8501). Open this URL in your browser to view and interact with the app.



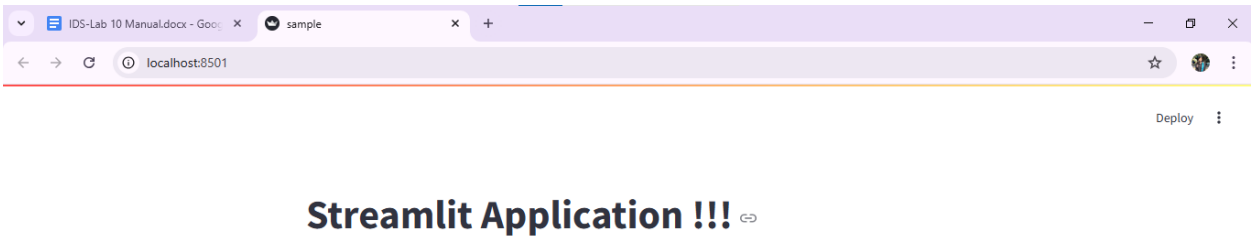
3.3 Understanding Streamlit basic functions

Streamlit offers simple built-in functions to create interactive web apps using Python. These functions help display text, add widgets, visualize data and handle user input.

3.3.1 Title

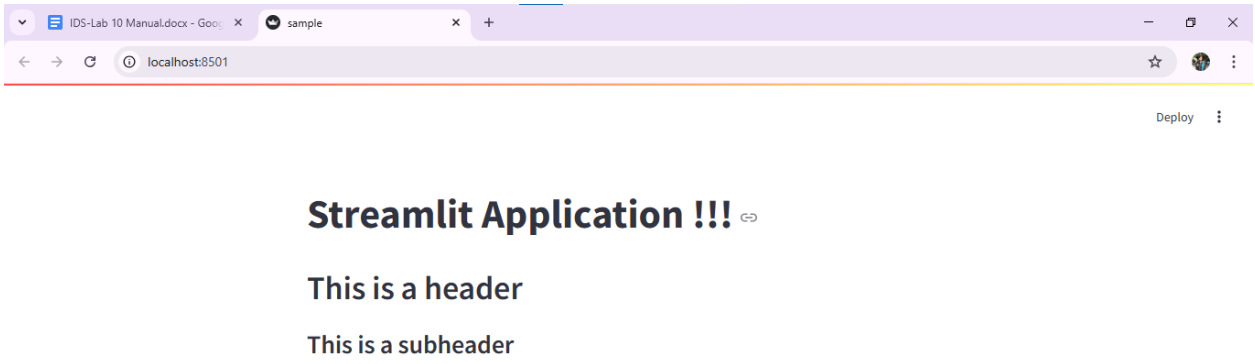
This basic Streamlit example displays a title on the web page, confirming that Streamlit is set up and running correctly.

```
import streamlit as st
st.title("Streamlit Application !!!")
```



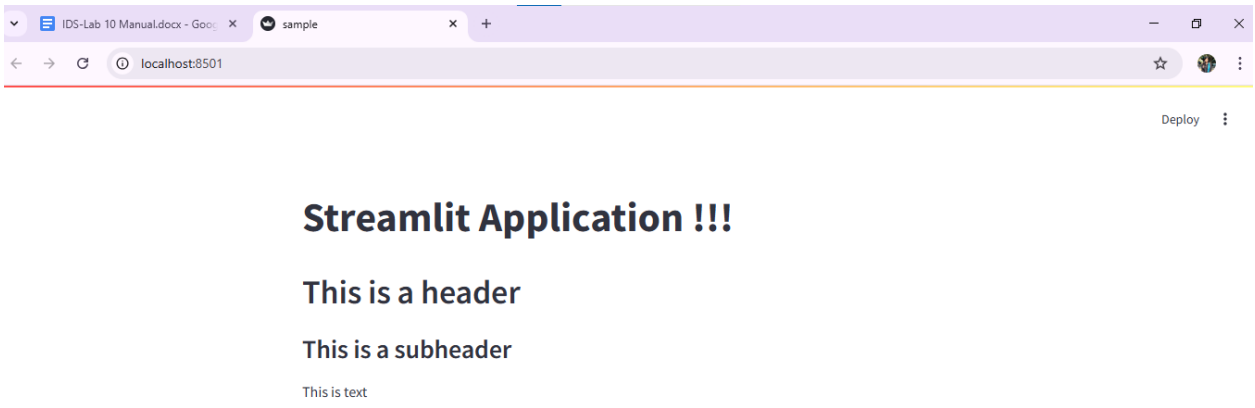
3.3.2 Header and Subheader

```
st.header("This is a header")
st.subheader("This is a subheader")
```



3.3.3 Text

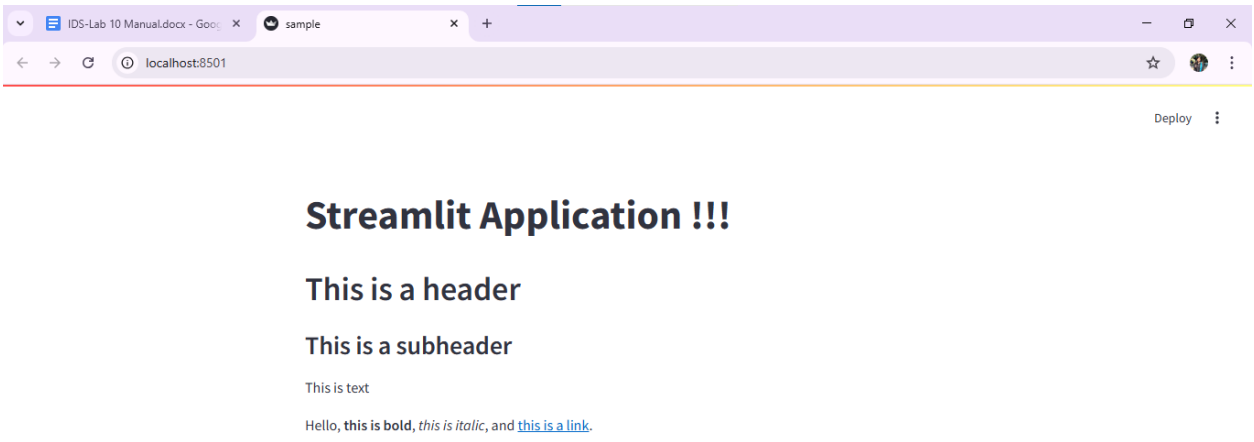
```
st.text("This is text")
```



3.3.4 Markdown

Markdown is a lightweight markup language used for rich text formatting. It supports bold, italic, headings, lists, links, images, and code blocks. It can also include HTML tags (if unsafe_allow_html=True). This is best for formatted content, documentation, or styled UI text.

```
st.markdown("Hello, **this is bold**, *this is italic*, and [this is a link](https://streamlit.io).")
```



3.3.5 Success, Info, Warning, Error and Exception

Streamlit app using built-in functions like st.success(), st.info(), st.warning(), st.error() and st.exception(). These functions help communicate status, information, warnings, errors and exceptions to users clearly.

```
st.success("Success")
st.info("Information")
st.warning("Warning")
st.error("Error")
exp = ZeroDivisionError("Trying to divide by Zero")
st.exception(exp)
```

Success

Information

Warning

Error

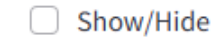
ZeroDivisionError: Trying to divide by Zero

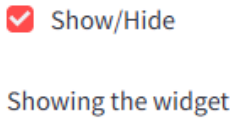
3.3.6 Display Images

```
from PIL import Image  # Import Image from Pillow
img = Image.open("streamlit.png") # Open the image file
st.image(img, width=200) # Display the image with a specified width
```

3.3.7 Checkbox

```
# Display a checkbox with the label 'Show/Hide'
if st.checkbox("Show/Hide"):
    # Show this text only when the checkbox is checked
    st.text("Showing the widget")
```

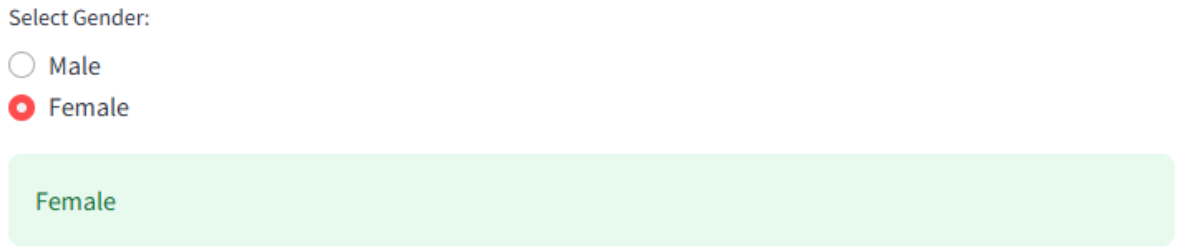




3.3.8 Radio button

```
# Create a radio button to select gender
status = st.radio("Select Gender:", ['Male', 'Female'])

# Display the selected option using success message
if status == 'Male':
    st.success("Male")
else:
    st.success("Female")
```

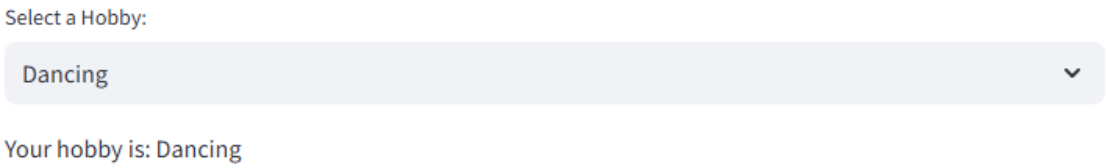


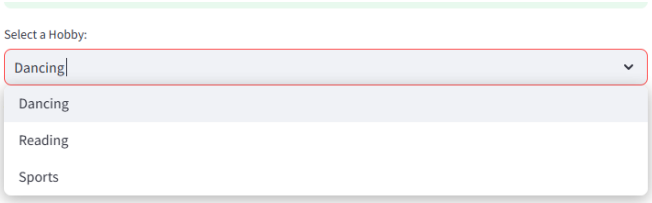
3.3.9 Selection Box

```
# Create a dropdown menu for selecting a hobby
```

```
hobby = st.selectbox("Select a Hobby:", ['Dancing', 'Reading',
'Sports'])

# Display the selected hobby
st.write("Your hobby is:", hobby)
```

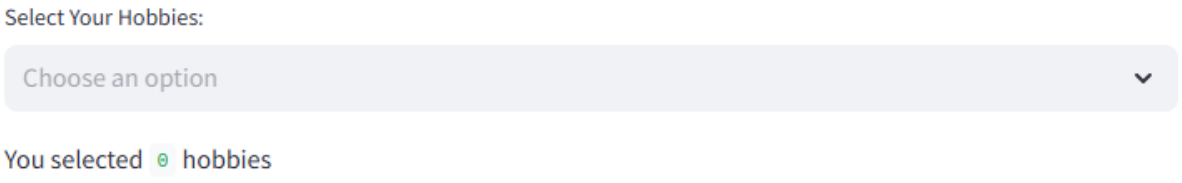


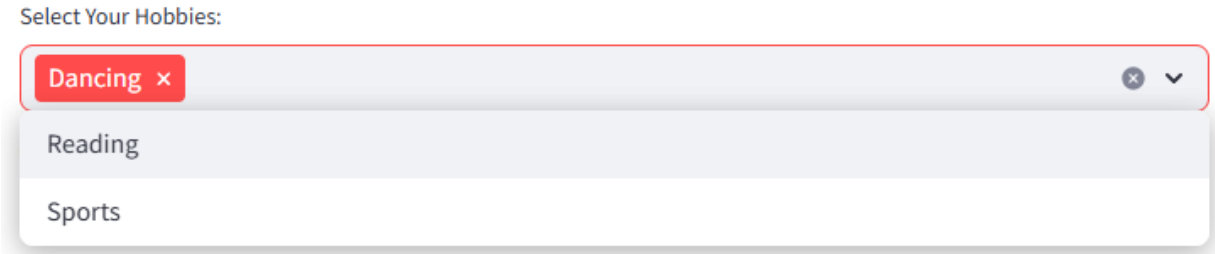


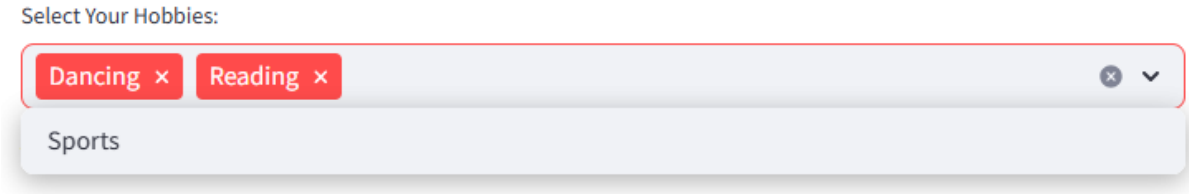
3.3.10 Multi-Selectbox

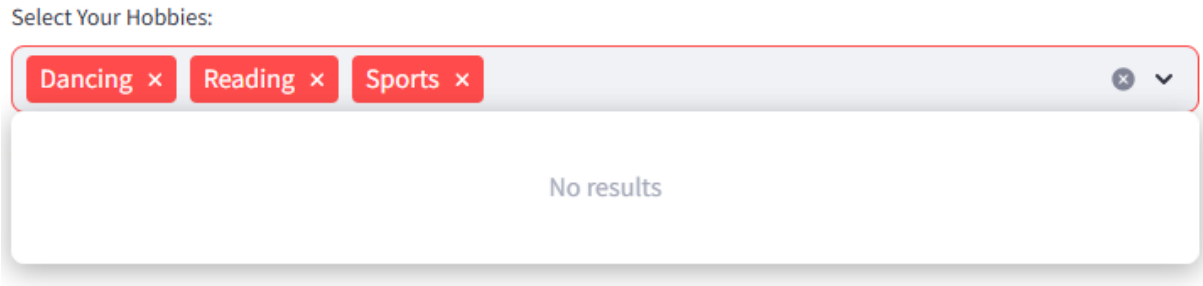
```
# Create a multiselect box for choosing hobbies
hobbies = st.multiselect("Select Your Hobbies:", ['Dancing', 'Reading',
'Sports'])

# Display the number of selected hobbies
st.write("You selected", len(hobbies), "hobbies")
```



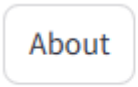


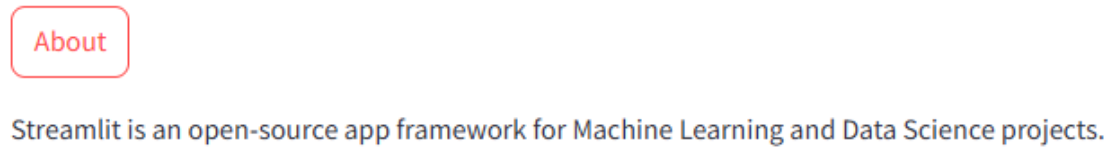




3.3.11 Button

```
# A button that displays text when clicked
if st.button("About"):
    st.text("Streamlit is an open-source app framework for Machine
Learning and Data Science projects.")
```

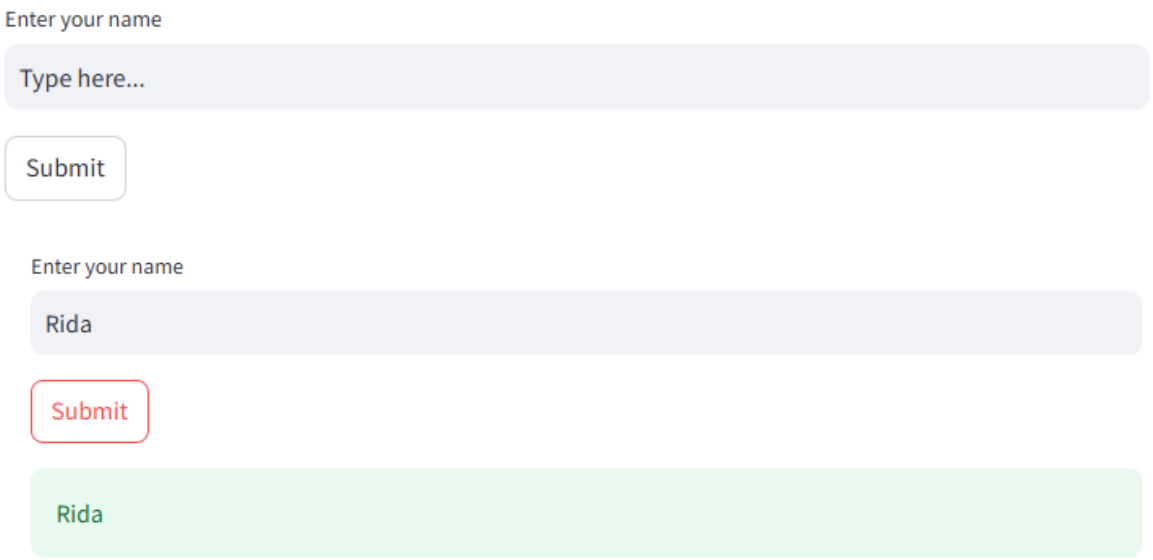




3.3.12 Text Input

```
# Create a text input box with a default placeholder
name = st.text_input("Enter your name", "Type here...")

# Display the name after clicking the Submit button
if st.button("Submit"):
    result = name.title()  # Capitalize the first letter of each word
    st.success(result)
```

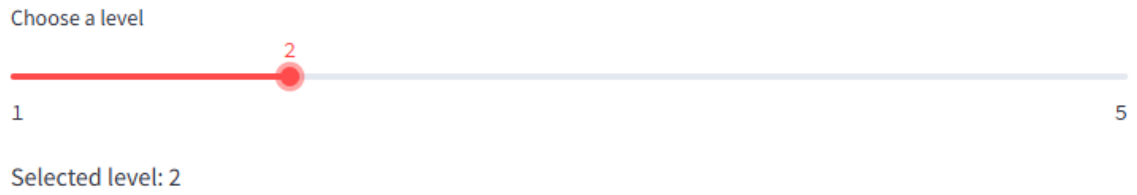


3.3.13 Slidebar

```
# Create a slider to select a level between 1 and 5
level = st.slider("Choose a level", min_value=1, max_value=5)

# Display the selected level
```

```
st.write(f"Selected level: {level}")
```



4 Streamlit Model Integration

We'll build and deploy a trained Scikit-learn model with Streamlit. You should have the following folder structure:

```
model_deployment/
│
├── iris_model.pkl           # saved trained model
├── app.py                   # streamlit app
├── requirements.txt         # dependencies
└── train_model.py           # model training script
```

4.1 Train and Save the Model

Create a file train.py

```
import pickle
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier

# Load dataset
iris = load_iris()
X, y = iris.data, iris.target

# Split data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)
```

```
# Train model
model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train, y_train)

# Save model
with open("iris_model.pkl", "wb") as f:
    pickle.dump(model, f)

print("Model trained and saved as iris_model.pkl")
```

Run this once to create the model file.

4.2 Streamlit Deployment Script

Create a file app.py

```
import streamlit as st
import pickle
import numpy as np

# Load trained model
with open("iris_model.pkl", "rb") as f:
    model = pickle.load(f)

# Page title
st.title("Iris Flower Classifier")
st.write("Predict the type of Iris flower using sepal and petal
measurements.")

# Sidebar inputs
st.sidebar.header("Input Features")
sepal_length = st.sidebar.slider("Sepal length (cm)", 4.0, 8.0, 5.4)
sepal_width = st.sidebar.slider("Sepal width (cm)", 2.0, 4.5, 3.4)
petal_length = st.sidebar.slider("Petal length (cm)", 1.0, 7.0, 4.3)
petal_width = st.sidebar.slider("Petal width (cm)", 0.1, 2.5, 1.3)

# Prepare input
features = np.array([[sepal_length, sepal_width, petal_length,
petal_width]])
```

```
# Predict
prediction = model.predict(features)[0]
class_names = ["Setosa", "Versicolor", "Virginica"]
predicted_class = class_names[prediction]

# Display
st.subheader("Prediction:")
st.success(f"The model predicts this flower is: **{predicted_class}**")
```

4.3 Running Locally

In your terminal:

```
streamlit run app.py
```

More information on streamlit: https://docs.streamlit.io/get-started/fundamentals/main-concepts

Exercise

1. You are provided with a dataset of 50 online shoppers (CSV given). Your task is to build a KNN-based purchase prediction system and deploy it through a Streamlit app.

1. Load the Dataset

Load the provided dataset into a pandas DataFrame.

2. Data Cleaning

- Handle missing values
- Remove duplicates
- Ensure correct data types

3. Encode Categorical Columns

Encode:

- `Device_Type` (Mobile / Desktop / Tablet)
- `Previous_Purchase` (Yes / No)
- `Region` (North / South / East / West)

You may use:

- LabelEncoder
- OneHotEncoder
- Or pandas `get_dummies()`

4. Train a KNN Classifier

Train at least 3 KNN models using different values of **k** such as:

- k = 3
- k = 5
- k = 7

Use scikit-learn's `KNeighborsClassifier`.

5. Evaluate All Models

Evaluate each model using:

- Accuracy
- Precision
- Recall
- F1-score

Select the best-performing KNN model.

6. Save the Best Model

Use the following code to save the trained KNN model:

```
import pickle

with open("knn_model.pkl", "wb") as f:

    pickle.dump(best_knn, f)   # best_knn is your chosen model
```

7. Build a Streamlit Application

Your Streamlit app must:

Display:

- Title
- Description
- Instructions

Load:

- The saved `knn_model.pkl`

Provide Input Widgets:

Use:

- Sliders (for Session_Duration, Pages_Visited, Bounce_Rate)
- Number inputs (for Time_On_Site)
- Radio buttons / select boxes (for Device_Type, Region, Previous_Purchase)

Predict:

- Whether the customer will purchase (1) or not (0)

Display Output:

Use:

- `st.success("The customer is likely to purchase.")`
- `st.error("The customer is not likely to purchase.")`

9. Submit:

You must submit the following:

1. **Jupyter Notebook (.ipynb)** showing:
    - Preprocessing
    - Model training
    - Evaluation
    - Saving model
2. **Streamlit app code (.py)**
3. **Saved model file (.pkl)**
4. **Two screenshots** of the Streamlit app:
    - One showing prediction = purchase
    - One showing prediction = not purchase
5. Zip all files into one with your name_rollnum.